

UASLP

Universidad Autónoma de San Luis Potosí

Universidad Autónoma de San Luis Potosí

Facultad de Ingeniería

202402300015

Área de Ciencias de la Computación

Interfaces Gráficas y Aplicaciones

Juego UNO.

Docente: Alberto Nuñez Varela

Estudiante(s):

Muñiz Rosas Rodrigo

Rojas Mendoza Rodrigo Javier

Ruiz Ortiz Jose Antonio

San Luis Potosí, S. L. P.

08 de noviembre del 2026

Semestre 2026-2027/I

Ingeniería en Sistemas Inteligentes	
Objetivo Educacional	**Atributo de Egreso**
OE1: Los egresados son profesionales que se desempeñan con éxito en la práctica de las Ciencias Computacionales o de las áreas relacionadas con las Tecnologías de la Información y la Comunicación.	**AE1**. Aplicará los principios de las matemáticas, ingeniería y de las ciencias computacionales para la solución de problemas complejos en ámbitos sociales, públicos o privados.
OE2: Los egresados son ingenieros competitivos a nivel nacional e internacional capaces de adaptarse a diferentes ámbitos de trabajo.	**AE3**. Desarrollará y conducirá investigación y experimentos utilizando el juicio ingenieril, analizando e interpretando datos a través del uso y generación de herramientas tecnológicas modernas; para con ellas establecer de manera adecuada límites y conclusiones del impacto de sus resultados en la sociedad, la economía, la sostenibilidad, la salud, la seguridad, los marcos legales y el medio ambiente.

Creación del proyecto: juego UNO.

Descripción General del Proyecto: Juego UNO de 3 Jugadores

Nuestro proyecto es una adaptación digital de escritorio del clásico juego de cartas UNO, diseñada específicamente para partidas locales de 3 jugadores bajo un formato de "asiento caliente" (hot-seat). El objetivo principal ha sido replicar la experiencia tradicional de juego de mesa en una interfaz fluida, interactiva y visualmente atractiva.

Tecnologías y Lenguajes Utilizados

Para la creación de este juego hemos utilizado C# como lenguaje de programación principal, apoyándonos en el entorno de .NET 8.0.

Para el desarrollo de la interfaz gráfica de usuario (GUI), decidimos implementar Avalonia UI. Elegimos este framework porque nos permite construir aplicaciones de escritorio modernas y eficientes empleando XAML para el diseño visual, ofreciendo además la ventaja de ser multiplataforma.

Para el almacenamiento de información se utilizó Mysql workbench, ya que nos permite hacer bases de datos relacionales de manera sencilla, así como proporciona una manera elegante de presentar los datos así como de poder ver los modelos entidad relación según las información de la base de datos.

Para la parte del backend se utilizó FastAPI para poder realizar las queries mediante una api, en cuanto a las librerías se emplearon SQLAlchemy, Pydantic, Uvicorn y PyMySQL que nos ayudaron a poder interactuar con la base de datos mediante código de Python.

A nivel de arquitectura de software, hemos estructurado todo el proyecto bajo el patrón MVVM (Model-View-ViewModel). Esto nos ha permitido separar de manera limpia y estricta la lógica central del juego (las reglas, el mazo, los jugadores) de la interfaz gráfica, haciendo que el código sea mucho más mantenible y escalable.

Arquitectura y Componentes

El proyecto se divide en cuatro pilares fundamentales:

Los Modelos (Models): Aquí reside el "cerebro" del juego. Tenemos clases que definen qué es una carta (Card), sus colores y valores, así como la entidad del jugador (Player) que almacena su mano de cartas. El núcleo es la clase UnoGame, que se encarga de aplicar las matemáticas y reglas del juego: inicializa el mazo, lo revuelve, gestiona la pila de descarte e implementa las mecánicas de las cartas de acción (Reversa, Salto, Toma Dos, etc.).

Las Vistas (Views): Diseñadas en XAML, representan el tablero interactivo. Hemos implementado un diseño inmersivo donde el jugador en turno siempre se ubica en la parte inferior de la pantalla, mientras que las manos de los oponentes se distribuyen en los laterales, utilizando transformaciones de rotación para dar la sensación de estar sentados alrededor de una mesa cuadrada.

El ViewModel: Actúa como puente de comunicación. Controla el flujo de la partida, actualiza los mensajes de estado en la pantalla y gestiona las acciones que hace el usuario (como robar una carta o terminar su turno), notificando a la interfaz visual de cualquier cambio al instante.

Los servicios: Es la vía de comunicación de nuestro programa C# con la base de dato en workbench, controla la solicitudes get y post que se le hacen desde C# a python para poder realizar las solicitudes a la api sobre la base de datos, así como el código necesaria de python para poder interactuar con la base de datos y para poder recibir las peticiones que se hacen desde C#.

Funcionamiento de la Partida

El flujo del juego está diseñado para dar control total a los jugadores. Al iniciar, el motor genera un mazo único (con una distribución estricta de una carta por número/color más comodines), reparten 7 cartas a cada uno de los 3 jugadores y voltea la primera carta en el centro de la mesa.

Durante su turno, el jugador activo puede evaluar su mano. Si tiene una carta que coincida en color, número o si posee un comodín, puede hacer clic en ella para jugarla. El sistema valida la jugada en tiempo real; si es correcta, la carta pasa a la pila de descarte. Si el jugador no tiene una carta válida (o elige no jugarla), puede hacer clic en el mazo principal para robar cartas a su mano.

Para mantener la dinámica del modo local, implementamos un sistema de control de turnos manual: una vez que el jugador ha terminado sus acciones, debe presionar la tecla Enter. Al hacerlo, el sistema rota virtualmente la mesa, pasando el control al siguiente jugador correspondiente, evaluando previamente si hubo algún efecto de "Salto" o "Reversa" por las cartas de acción jugadas.

División de equipo.

Para la división de equipo nos hemos encargado de seguir la siguiente estructura en la medida de lo posible:

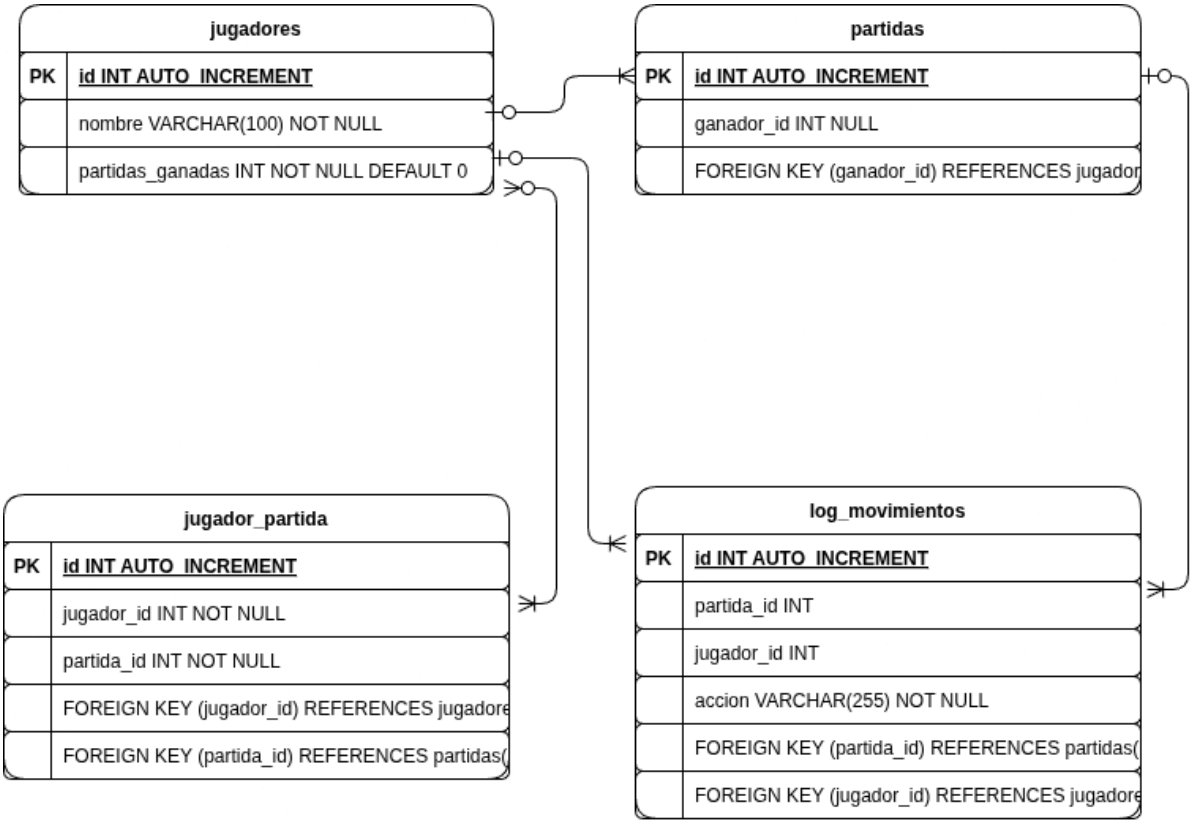
- **Rosas Muñiz Rodrigo:**
    - Como responsable del núcleo y las mecánicas, su rol ha consistido en dar vida al motor interno del juego y asegurar que cada regla se cumpla a la perfección.
    - Ha construido los modelos estructurales en C# (UnoGame, Card, Player), programando la lógica de validación para determinar qué cartas son jugables según el estado de la mesa.
    - Gran parte de su trabajo se ha enfocado en la resolución de problemas técnicos y la gestión del flujo de los turnos, manejando la matemática detrás de las cartas de acción (saltos de turno, reversas y acumulación de cartas) y garantizando la correcta comunicación entre la vista y los datos.
    - En resumen, se encarga de que el juego sea estable, mantenga reglas justas y opere sin interrupciones.

- **Rojas Mendoza Rodrigo Javier:**
    - Su trabajo se ha centrado en diseñar y construir la cara visible de la aplicación, asegurando que la experiencia de los jugadores sea inmersiva y fiel al juego de mesa tradicional.
    - Utilizando el framework Avalonia UI y el lenguaje de diseño AXAML, ha estructurado la ventana principal (MainWindow), organizando el tablero de juego para que el jugador activo siempre se posicione al centro, mientras que las manos de los oponentes se distribuyen en los costados mediante el uso de transformaciones visuales.

- Además, ha sido responsable de enlazar estas vistas con la lógica subyacente mediante el patrón MVVM, logrando que la interfaz se actualice en tiempo real durante la partida. Sus próximos pasos incluirán refinar la interfaz visual para la selección de colores en los comodines y diseñar las pantallas de victoria.

- **Ruiz Ortiz Jose Antonio:**
  - La contribución de este integrante se enfoca en la siguiente gran fase de evolución del proyecto: la persistencia de datos y la conectividad.
  - Dado que el juego base ya es funcional de manera local, su trabajo consistirá en diseñar e implementar toda la arquitectura de backend.
  - Está a cargo de construir una API y estructurar una Base de Datos relacional que permitirá llevar un registro detallado de las partidas. A través de este sistema, el proyecto podrá almacenar perfiles de jugadores, así como guardar un historial de las acciones realizadas en las partidas.
  - Su objetivo principal es que toda la información generada quede respaldada de forma estructurada y segura, sentando además las bases técnicas para una futura escalabilidad del sistema.

Diagrama Entidad-Relación:



Descripción de la API.

API Backend para Juego de UNO: Sistema backend desarrollado en FastAPI diseñado para gestionar la lógica de partidas, el registro de jugadores, el control de turnos/movimientos y las estadísticas de victoria de un juego de cartas de UNO integrado con una interfaz cliente en C#.

Tecnologías utilizados:

Se utilizaron principalmente 4 tecnologías:

- Framework Web(FastAPI)) para la creación de la API rendimiento
- Base de datos: MySQL para poder tener una base de datos relacional
- Control de Versiones: Git/Git hub

Arquitectura y Modelo de datos.

Arquitectura Cliente-Servidor: El sistema está dividido en dos grandes bloques independientes que no comparten código, y se comunican mediante el protocolo HTTP.

La relación entre ambos mundos se basa en un flujo de petición y respuesta HTTP (Request-Response) a través de servicios de red:

La aplicación en C# actúa como el Cliente. Es la encargada de mostrar las pantallas del juego de UNO, manejar los eventos de los usuarios, controlar la baraja local y decidir cuándo ocurre una acción (como jugar una carta, robar o ganar). Mediante clases de servicio HTTP como el ApiService, la aplicación serializa los datos de la jugada a formato JSON y realiza peticiones de tipo POST o GET hacia los puertos donde corre el servidor de Python.

FastAPI actúa como servidor, escuchando las peticiones de C#. Se, utiliza Pydantic para tomar el JSON que manda C#, verificar que los tipos de datos sean correctos y rechazar cualquier formato erróneo automáticamente. Una vez validada la petición, utiliza SQLAlchemy para traducir esas acciones en consultas SQL seguras hacia la base de datos. Finalmente, procesa el resultado y devuelve una respuesta en formato JSON de vuelta a C# como un mensaje de éxito, el ID de una nueva partida o los datos de un usuario.

Para poder guardar la información solicitada se utilizaron 3 tablas, las cuales contaban con relaciones entre sí,las cuales son:

- jugadores:Almacena la información de los participantes y su contador de partidas ganadas
- partidas: Registra el estado de cada juego, vinculando al jugador y al ganador de la partida(permitiendo valores nulos para juegos truncados o inconclusos)
- log_movimiento: Funciona como un historial detallado de las acciones y jugadas realizadas durante cada partida.

Endpoints:

Para poder guardar la información solicitada se utilizaron 3 tablas, las cuales contaban con relaciones entre sí,las cuales son:

- POST /jugadores/: Recibe el nombre de un nuevo jugador mediante un esquema de Pydantic y lo registra formalmente en la base de datos, retornando el objeto con su ID generado.
- GET /jugadores/: Consulta y devuelve una lista con todos los jugadores registrados actualmente en la base de datos.
- POST /jugadores/login: Gestiona el inicio de sesión buscando a un jugador por su nombre; si ya existe, devuelve sus datos existentes, y si no, lo registra automáticamente con 0 partidas ganadas.
- POST /movimientos/: Permite registrar de forma genérica cualquier movimiento o acción detallada especificando el ID de la partida, del jugador y el texto del movimiento.
- GET /partidas/{partida_id}/log: Consulta y retorna todo el historial completo de acciones y movimientos registrados asociados al ID de una partida específica.
- POST /partida/nueva: Inicializa y registra una nueva partida en la base de datos utilizando el autoincremento, vinculando opcionalmente un anfitrión y retornando el ID generado automáticamente.
- POST /partida/robar-carta: Registra en el log de movimientos cuando un jugador roba una carta del mazo, especificando el color y el valor de la tarjeta obtenida.
- POST /partida/tomar-penalizacion: Añade una entrada al historial indicando que un jugador recibió y aceptó una penalización de cartas.

- POST /partida/pasar-turno: Deja constancia en el log de movimientos cuando un jugador decide ceder o pasar su turno sin tirar.
- POST /partida/jugar-carta: Procesa los movimientos y acciones de los jugadores durante el desarrollo del juego, evaluando si se jugó un comodín (Wild) para registrar el cambio de color elegido.
- POST /partida/uno: Registra la decisión del jugador al presionar el botón de "UNO", especificando si gritó la advertencia a tiempo (True) o si omitió hacerlo (False).
- POST /partida/sin-cartas: Deja constancia en el registro de la partida cuando el mazo principal de juego se agota.
- POST /partida/ganó: Registra el final de la partida, actualiza el contador de victorias del jugador ganador, marca el ID del ganador en la tabla de partidas y añade la entrada correspondiente al log de movimientos.

Organización de las tareas:

21 de agosto(Javier): realizará el primer commit como era de esperarse para poder empezar a trabajar con el proyecto, además de que se agregaran las primeras imágenes esenciales para la visualización del programa

22 - 28 de agosto(equipo): Planeación de la arquitectura a utilizar, flujo de trabajo a realizar, modelos de cartas, lluvia de ideas sobre la parte visual, mecánicas de juego, determinación de las reglas, investigación de librerías que hacen falta, así como fechas de realización y entrega.

29 de agosto(Rodrigo): Creación de modelos visuales, así como de lógica de programación sobre la interacción de las cartas y su uso.

30 de agosto(Rodrigo): Pruebas del código del entorno visual, sobre el menú principal, y corrección de errores

1 de octubre(Rodrigo): Revisión de errores sobre la vista de juego hacia los demás jugadores para poder visualizar una parte de las cartas de los demás jugadores y se pueda hacer la rotación entre jugadores, además terminar con las funcionalidades principales

1 de octubre(Javier): Autorizar cambios sobre la vista del juego realizados por rodrigo y hacer merge, además de arreglar los fallos que se produjeron y agregar funcionalidades de jugabilidad

2 de octubre(Antonio): Creacion de modelos de tablas en python, y archivo main para endpoints principales de la API

2 de octubre(Rodrigo): Revisión de errores de las funciones principales y agregar cambios para las funciones de UNO oficiales acordadas

5 de octubre(Javier): Refinaciones a la interfaz con modelos que tomaron más tiempo preparar

6 de octubre(Antonio): Realizar comentarios al código para que los compañeros puedan comprender la parte de la base de datos

6 de octubre(Javier): Agregar imágenes más detalladas con aumento de resolución,

7 de octubre(Antonio): Creacion de clases para sincronizar llamadas de API con C#

8 de octubre(Antonio): Creacion de endpoints para creación de jugadores y sincronización, además de endpoints para principales acciones realizadas por las cartas, además de agregar el Readme básico para ejecutar el repositorio

9 de octubre(Antonio): Corrección de errores sobre el código que relaciona base de datos con python y C# con python, además de adición de endpoints para más funcionalidades relacionadas con el registro de victorias.

11 de octubre(Antonio): Últimas modificaciones y agregación de función extra para tener un registro de ganadores de cada partida, además del registro del documento final

Relación de tareas y commits:

Fecha	Miembro	Tarea Reportada	N° y Mensaje del Commit
21 Sep	Javier	Primer commit e imágenes iniciales. *(Nota: Reportado como 21 Ago)*	**#1** Primer commit(Pantallas e imagenes).
22 - 28 Ago	Equipo	Planeación, arquitectura, mecánicas y flujo de trabajo.	*(Fase de diseño - Sin commits asociados)*
29 Sep	Rodrigo	Modelos visuales y lógica de interacción de cartas. *(Rep. 29 Ago)*	**#2** Ajuste los Modelos y la logica del juego...
30 Sep	Rodrigo	Pruebas visuales y configuración del menú principal. *(Rep. 30 Ago)*	**#3** Prueba de como se ve el juego **#4** Ya se ve el menu principal... **#5** Hice unos ajustes en la vista...
1 Oct	Rodrigo	Ajuste de vista multijugador, rotación de turnos y funciones base.	**#6** Esta ajustada la vista de los jugadores... **#10** Termine con las funciones principales...

1 Oct	Javier	Revisión de vistas (Merge), corrección de fallos y jugabilidad.	**#7** Merge VistaJuego **#8** Commit despues del merge **#9** Arreglo de fallos producidos...
2 Oct	Rodrigo	Sincronización de interfaz y corrección de reglas (carta Wild).	**#11** Arregle un bug de la carta wild... **#12** Sincronizar cambios finales de interfaz...
2 Oct	Antonio	Creación de modelos de tablas BD y archivo main de API.	**#13** Conexion con la base de datos **#14** Modelos de tablas con interfaz... **#15** Archivo main... **#16** Main corregido version 2
5 Oct	Javier	Refinaciones a la interfaz con modelos detallados.	**#17** Arreglos y adiciones a la interfaz.
6 Oct	Antonio	Documentación y comentarios sobre BD y API para el equipo.	**#18 - #20** Comentarios funcionamiento de conexion / API C# / Modelos **#22** Merge API_BD

6 Oct	Javier	Agregado de imágenes detalladas en alta resolución.	**#21** Modificaciones a detalles de la interfaz.
7 Oct	Antonio	Clases de API, obtención de datos y sincronización local/BD.	**#23 - #28** Archivos temporales, imagen uno.png, integracion BD, Servicio API, funcion datos.
8 Oct	Antonio	Endpoints de login, conexión de jugadores y acciones de cartas.	**#29 - #32** Endpoints login/jugadores, acciones robar, penalizar y turno, relacionar API.
9 Oct	Antonio	Correcciones de DB/Models, endpoints de victoria/UNO y registros del log del juego.	**#33 - #49** Correcciones conexion/models/main, endpoints victoria, logs de partida/mazo.
9 oct	Javier	Agregado de botones a la interfaz (Opciones y UNO) y corrección de imágenes.	**#50** Agregado de boton Opciones. -RJ. **#51** Agregado de boton uno e imagenes corregidas. -RJ.
10 Oct	Antonio	Estabilización y merge de ramas C#.	**#52** Merge pull request #8 API_C#
11 Oct	Antonio	Partidas en BD, registro de ganadores y Merge final del proyecto.	**#53 - #58** Agregar partidas a BD, corrección errores main/partida nueva, Merge pull request #9 API_C#.

Listado de Commits *(Con fecha 11 de octubre de 2026).*

Usuarios	**Alumno**
javydhu	Rojas Mendoza Rodrigo Javier
RodrigoMunizRosas1	Muñiz Rosas Rodrigo
jaro554	Ruiz Ortiz José Antonio

commit 9abaf543de1e88d42f0e9c2daa0e21c834ffec0e (HEAD -> main, origin/main, origin/HEAD)
Author: javydhu <javierojas358@gmail.com>
Date: Sun Nov 8 23:28:36 2026 -0600

Ultimos detalles a interfaz visible. -RJ.

commit b3dc21d4fe2c88194655704dc4f8c583af1a888c (origin/ConexionComputadoras)
Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>
Date: Sat Nov 7 23:57:31 2026 -0600

Ya se conecta correctamente los jugadores con el servidor (la partida)

commit e7e21e624cf2ab529aad3ab9a7c5034436602e3c
Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>
Date: Sat Nov 7 23:04:28 2026 -0600

Creacion de la conexion de de diferentes computadoras

commit 7b10791e2e5c30e17b695302f4b4725c45f9b67e
Author: javydhu <javierojas358@gmail.com>
Date: Fri Nov 6 23:52:26 2026 -0600

Correcion Errores. Actualizacion Variables. Optimizacion Compilacion. -RJ.

commit cf5c2b8a813806f6219a2eeb50ea0bf4b7dcfaf9

Merge: a726a31 11ab378
Author: jaro554 <jaromusicion@gmail.com>
Date: Wed Nov 4 13:22:56 2026 +0000

  Merge pull request #10 from javydhu/Nueva_Base

  Modificacion a base de datos

commit 11ab3780af8e6aeae9f3b777624f784b31d7e12f
Author: Antonio <jaromusicion@gmail.com>
Date: Wed Nov 4 07:20:00 2026 -0600

  End point para registrar jugadores agregado

commit acf4445e130f334dfd8be618e05f7a92432048cb
Author: Antonio <jaromusicion@gmail.com>
Date: Wed Nov 4 07:06:47 2026 -0600

  Cambio de la base de datos

commit 87bc9b51389cc5f6d626363c1884a3fb0bbdbcbb (HEAD -> main, origin/main)
Merge: c30cad0 d0259bd
Author: jaro554 <jaromusicion@gmail.com>
Date: Mon Oct 12 01:11:46 2026 +0000
  Merge pull request #9 from javydhu/API_C#
  Api c#

commit d0259bd893d45233cd92163684496b1b36b99773 (origin/API_C#)
Author: jaro554 <jaromusicion@gmail.com>
Date: Sun Oct 11 19:00:07 2026 -0600
  Correccion de errores en funcion partida nueva, dato no existente

commit 6d635d5954bd3c4de840ee3ac6f7b4a67d104880
Author: jaro554 <jaromusicion@gmail.com>

Date: Sun Oct 11 18:41:19 2026 -0600

Correccion de errores y actualizacion de dueño de la partida

commit bec309f22e84a88a9e94dcb467590a0334985902

Author: jaro554 <jaromusicion@gmail.com>

Date: Sun Oct 11 18:06:25 2026 -0600

Correccion de nombre de variable mal llamada

commit 17752c0ef4c17e72fd86d5b5e7d56f657e657748

Author: jaro554 <jaromusicion@gmail.com>

Date: Sun Oct 11 17:59:33 2026 -0600

Correccion de errores en main.py

commit 4a54e5cabfd51e2a9e5fb645abc10a7850cb731e

Author: jaro554 <jaromusicion@gmail.com>

Date: Sun Oct 11 17:54:09 2026 -0600

Agregar partidas a la base de datos

commit c30cad0220d4e9963174dafb2410ee9a5e25c7bc

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 21:28:51 2026 -0600

Agregar victoria por puntos en el log

commit 555536194363ac73bbdeeaf6fab65f9d87b2fd9a

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 21:03:38 2026 -0600

Agregar mensaje de mazo sin cartas en el log

commit c2388987f8d12f3ca4238af4c05b7a7b47b1d624

Merge: 1dce6d5 f766c88

Author: jaro554 <jaromusicion@gmail.com>

Date: Sat Oct 10 01:18:38 2026 +0000

Merge pull request #8 from javydhu/API_C#

Api c#

commit f766c887983dafc057351d5ddeb748bbf6dd0f17
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 19:16:06 2026 -0600

    Agregar movimientos preparativos del juego al log

commit 44cf2fb4731ec712cd523081a31208f19278e8dd
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 12:47:55 2026 -0600

    Inicio del juego en el log

commit 1dce6d55018ba0101979485877c89c97ef7a2dce
Merge: 5fbda4b 53397c9
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 16:55:34 2026 +0000

    Merge pull request #7 from javydhu/API_C#

    Endpoint para registrar UNO agregado

commit 53397c9bd2d092654c5486897cdffa7f65da9eb0
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 10:46:50 2026 -0600

    Endpoint para registrar UNO agregado

commit 5fbda4b88febf7cf328c75f4ad613b9624b2b265
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 09:26:16 2026 -0600

    Agregado de boton Opciones. -RJ.

commit 5875da56695d72680f269035218d059377180bce
Author: v <jaromusicion@gmail.com>
Date: Fri Oct 9 09:15:02 2026 -0600

    Agregado de boton uno e imagenes corregidas. -RJ.

commit 8f2e26d18bcc09c2d84bd7a474c4429b92d6b339

Merge: 58d2dc6 986e03d

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 12:38:11 2026 +0000

Merge pull request #6 from javydhu/API_C#

Api c# cambios no guardados

commit 986e03d600aec1e1f95a169a231ba1a8f7e9a03b

Merge: 5ce6a5a 9c87b68

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 06:30:27 2026 -0600

Merge branch 'API_C#' of https://github.com/javydhu/UNO- into API_C#

commit 5ce6a5ade91bfaadbdd021e756fcf64d66787896

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 06:26:18 2026 -0600

Ultimas modificaciones a main.py

commit 58d2dc66ac0a3d0e66c9aa1e7822108c7aff7b5d

Merge: 8b9bb63 9c87b68

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 12:25:17 2026 +0000

Merge pull request #5 from javydhu/feature/logica-api-uno

Feature/logica api uno

commit 9c87b68f59ffe256d9f2a35d11e0fced26ccd4fa (origin/feature/logica-api-uno)

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 06:11:33 2026 -0600

Implementacion de victoria y correccion de errores en la conexion de la api con C#

commit d50fef7116dfabf64a5922dfe651ea961a52e30f

Author: jaro554 <jaromusicion@gmail.com>

Date: Fri Oct 9 05:23:05 2026 -0600

Agregar endpoint victoria e intregracion de sistema de cuenta de partidas

commit 763c994dee11e1a8983eb79dae18bdb804e26ec3
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 03:56:43 2026 -0600

    Correccion de errores en la conexion

commit 5a9fa49daa49f67ffb975e2dd1b54c750307d77a
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 03:36:48 2026 -0600

    Correccion de retorno de endpoints post

commit c0043eafd7aec174a3f9b53daa3f6a1e92c0e449
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 03:15:56 2026 -0600

    Correccion de errores en database

commit b46dae63ba78a80eab90a8c4f889d0113b00eb13
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 03:15:41 2026 -0600

    Correccion de errores en models

commit 63038a59c868e6147f0d7ec4e66ce0c8fe4f8d20
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 9 03:15:17 2026 -0600

    Correccion de errores en el Main

commit e9d025df357e761a36d5c5f08bbfccb4202f7a52
Author: jaro554 <jaromusicion@gmail.com>
Date: Thu Oct 8 07:16:11 2026 -0600

    Relacionar funciones de api con funciones del juego

commit fcb750600a3b58cfe2d9dac79b0f802df3348557
Author: jaro554 <jaromusicion@gmail.com>
Date: Thu Oct 8 07:07:17 2026 -0600

    acciones de robar, penalizar y turno agregadas para verse en base de datos

commit 80b358afb265058c7b0da8e9d4388f70477fc734
Author: jaro554 <jaromusicion@gmail.com>
Date: Thu Oct 8 06:36:01 2026 -0600

    Conexion de API con jugadores

commit 70886277421423db70c527c78d7c0d0ccd2cabab
Author: jaro554 <jaromusicion@gmail.com>
Date: Thu Oct 8 05:34:50 2026 -0600

    nuevo endpoint para login de jugadores

commit bb225516a5f6b345d4d423979cf80842eff198af
Author: jaro554 <jaromusicion@gmail.com>
Date: Wed Oct 7 22:51:05 2026 -0600

    Fucion API para obtener datos

commit 6988a5125435b867cc9e899c42206b1d76a98497
Author: jaro554 <jaromusicion@gmail.com>
Date: Wed Oct 7 22:17:19 2026 -0600

    Creacion de Servicio de API

commit 1da629d886c3a3149474c975f43a6fa59d135df0
Author: jaro554 <jaromusicion@gmail.com>
Date: Wed Oct 7 20:29:03 2026 -0600

    Comienzo de integracion de codigo con base de datos

commit 8b9bb636f718ca273dbfccc751a5abd28d886090
Author: jaro554 <jaromusicion@gmail.com>
Date: Wed Oct 7 05:25:48 2026 -0600

    Proyecto compilable despues cambios

commit d28dd28edbfd2c1ee7eb260720c7466c80a29f0c
Author: jaro554 <jaromusicion@gmail.com>
Date: Wed Oct 7 05:22:20 2026 -0600

Agregar imangen de uno.png

commit fb8e69859b847d78f29f8fb5d2d92599cabf4200
Author: jaro554 <jaromusicion@gmail.com>
Date: Wed Oct 7 05:13:26 2026 -0600

wip: guarda archivos temporales locales

commit f11c1e1e91dfb58435eb9bc2308288df2e9f181c (HEAD -> main)
Author: javydhu <javierojas358@gmail.com>
Date: Tue Oct 6 23:04:05 2026 -0600

Modificaciones a detalles de la interfaz. -RJ.

commit 9cec79c62b97d42aaaf6743fe89ff40360294acb
Merge: b4fd1c2 c7c48a4
Author: jaro554 <jaromusicion@gmail.com>
Date: Tue Oct 6 23:43:33 2026 +0000

Merge pull request #3 from javydhu/API_BD

Agregar api a programa

commit c7c48a42d3b0c46489b7196e9fc13900189edcee (origin/API_BD)
Author: jaro554 <jaromusicion@gmail.com>
Date: Tue Oct 6 17:32:17 2026 -0600

Clase modelos con comentarios de explicacion

commit df5413a27479c9383c2095ea915650d7d414c45d
Author: jaro554 <jaromusicion@gmail.com>
Date: Tue Oct 6 17:26:56 2026 -0600

Comentarios relacionados con el funcionamiento de la api con c#

commit 70bf3c88650b984fa7e190218d38164a9c11202b
Author: jaro554 <jaromusicion@gmail.com>
Date: Tue Oct 6 17:25:52 2026 -0600

Comentarios funcionamiento de la conexion con la base de datos

commit b4fd1c20d79921ade422247228e70c597d65f939
Author: javydhu <javierojas358@gmail.com>
Date: Mon Oct 5 13:42:15 2026 -0600

Arreglos y adiciones a la interfaz. -RJ.

commit d713d71ddee773ce576534174d8d2ee735ae0429
Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>
Date: Fri Oct 2 08:35:00 2026 -0600

sincronizar cambios finales de interfaz y reglas de uno

commit f9dae44e19647b7cda778056800e8aad5ca7dba1
Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>
Date: Fri Oct 2 07:12:42 2026 -0600

Arregle un bug de la carta wild al momento de cambiar a otro color

commit 031dc71d319f22b3ada9b31fdc281538c8aea60c
Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>
Date: Thu Oct 1 23:12:48 2026 -0600

Termine con las funciones principales del uno

commit 101462d2d6f6e9fbc6a732578c0a0c6091183b69
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 2 13:17:02 2026 -0600

Main corregido version 2

commit 0650f7302a0b586be511fdcd273b8c08294b3d66
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 2 13:12:00 2026 -0600

Archivo main que controla la api creado

commit ad06f257a11c237dc8026a3fda2a1beb24318619
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 2 12:52:34 2026 -0600

Modelos de tablas con interfaz de python hechos

commit 820d3e1408ad1f84866f20cf65a247c5ffb9f188
Author: jaro554 <jaromusicion@gmail.com>
Date: Fri Oct 2 12:35:48 2026 -0600

Conexion con la base de datos

commit c61c1db0c442524a933a41d74b4d1b86eef7cfd2
Author: javydhu <javierojas358@gmail.com>
Date: Thu Oct 1 19:18:05 2026 -0600

Arreglo de fallos producidos y funcionalidades de la jugabilidad agregadas. -RJ.

commit c14920e9d73ab420b31233a7e59ee8c551787c7d
Author: javydhu <javierojas358@gmail.com>
Date: Thu Oct 1 12:26:51 2026 -0600

Commit despues del merge

commit efff67298c0442b68ec0dbcfd268902f00e53c2b
Merge: 916b010 611134d
Author: javydhu <159716151+javydhu@users.noreply.github.com>
Date: Thu Oct 1 10:08:28 2026 -0600

Merge pull request #2 from javydhu/VistaJuego

Vista juego

commit 611134dcdce9c73743afa49f18b2a60529f2445b (origin/VistaJuego)
Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>
Date: Thu Oct 1 08:11:57 2026 -0600

Esta ajustada la vista de los jugadores y la rotacion del jugador en turno

commit 74af6e893574315a59c36dde0c1008673837cbeb
Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>
Date: Wed Sep 30 22:54:09 2026 -0600

Hice unos ajustes en la vista del juego

commit e6e1cda01bd168172c1c4530293ef0e5c3a21d5a

Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>

Date: Wed Sep 30 22:35:56 2026 -0600

Ya se ve el menu principal y el menu del juego

commit f1f10a2b8a9d71bb25287a5076434a8d2f29469a

Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>

Date: Wed Sep 30 13:35:35 2026 -0600

Prueba de como se ve el juego de uno

commit 40b9a017e828b37fb900f9058b158ec4ac31d89f (origin/ModelosLogica)

Author: RodrigoMunizRosas1 <a363427@alumnos.usalp.mx>

Date: Tue Sep 29 19:49:17 2026 -0600

Ajuste los Modelos y la logica del juego junto con el uso de las cartas

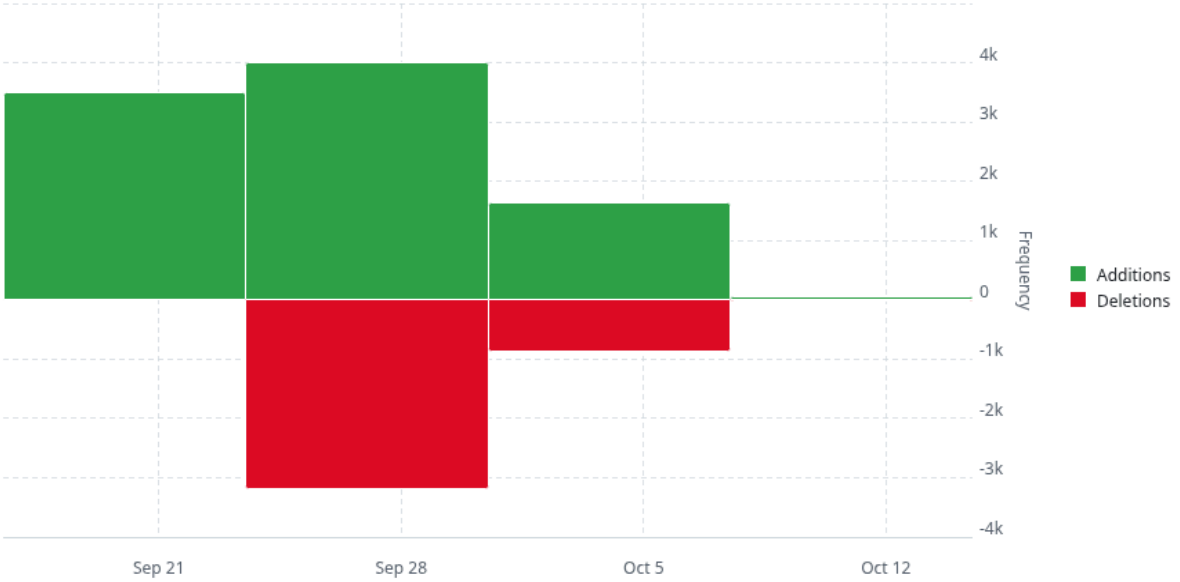
commit 916b010b0eefee7722c33db573d349ac51eaa7e1

Author: javydhu <javierojas358@gmail.com>

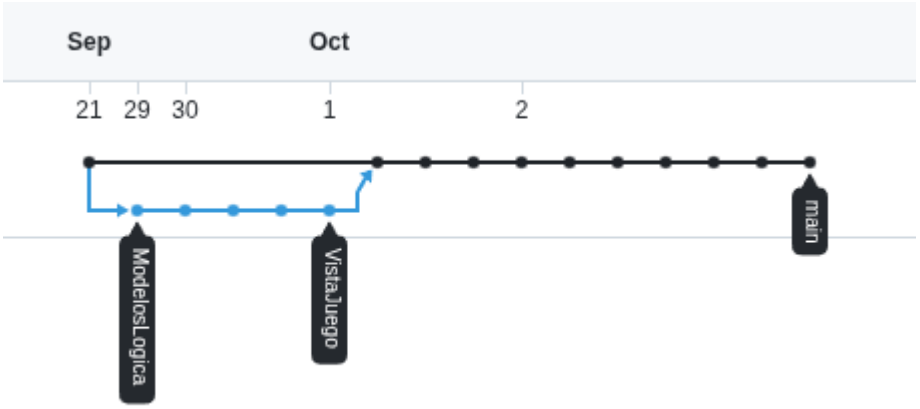
Date: Mon Sep 21 12:20:31 2026 -0600

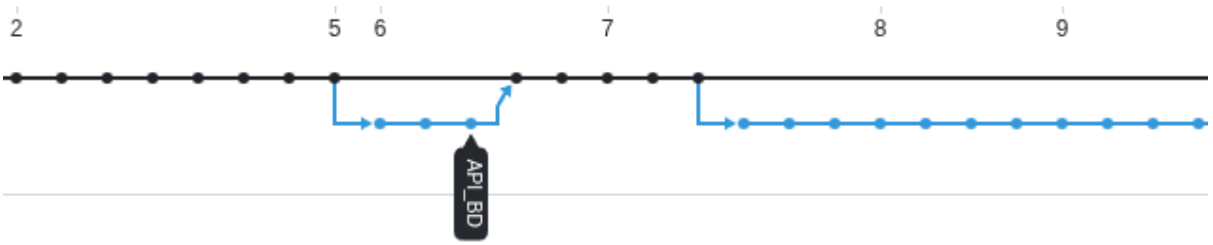
Primer commit(Pantallas e imagenes). -RJ.

Diagrama de additions and deletions en el transcurso del trabajo.

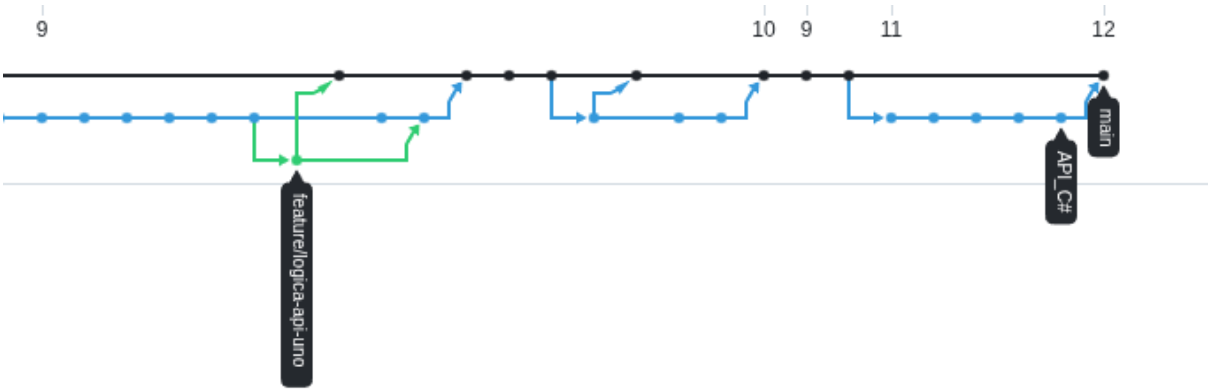


Listado de ramas por cronología.





+



Problemas encontrados.

Sobre los problemas encontrados tuvieron distintos orígenes, desde lo que son problemas con el manejo de versiones, con el acostumbrarse a la interfaz, la programación en distintos lenguajes de programación, y a los comandos de git hub, además de problemas con dispositivos propios e incluso de almacenamiento de memoria.

Github es una herramienta esencial para todos los programadores, nos permite colaborar con otros desarrolladores en la creación de proyectos colaborativos a gran escala, es un controlador de versiones de código y se deben de tener cuidados para poder trabajar con él, como lo es no trabajar en el mismo archivo y modificar demasiadas cosas, ya al momento de hacer merge con las ramas se pueden perder partes del código, o se pueden tener conflictos con el mismo, este problema no se presentó tan frecuentemente debido a que coincidimos que sería una parte peligrosa el modificar el mismo código y debido a eso programamos la mayoría de las cosas en diferentes días, o si teníamos que programar el mismo dia, no programamos el mismo archivo.

Otro problema que tuvimos con el manejo de versiones es tener buenas prácticas al momento de modificar código en una misma computadora, ya que es acostumbrado hacer pull a las ramas que vamos a modificar para asegurarnos que tengamos las versiones más recientes para poder modificarlas, principalmente con el main, cosa que muchas veces se pasaba por alto y se terminaba modificando una versión anterior, perdiendo tiempo.

Por otro lado, un integrante tuvo problemas con el sistema operativo windows, en el que estaba trabajando el proyecto, debido a configuraciones del sistema que le impiden trabajar en Windows, es por eso que decidió pasar su entorno de trabajo a linux incluyendo la base de datos e IDEs, programando de esta manera la parte de la API en Visual Studio Code y el código C# en Rider, lo cual a su vez le traería problemas con la memoria de su dispositivo, debido a que estos dos programas consumen demasiado cache llegando incluso a agotar la memoria del dispositivo en su totalidad y forzando al integrante a tener que borrar desde el entorno no gráfico estos archivos para poder utilizar su computadora.

Otro problema que se suscitó fue la fecha modificada en el repositorio debido a ubuntu, debido a que los últimos commits se vieron fuera de tiempo debido a que la computadora tenía internamente la hora global a pesar de que visualmente se veía la hora local, es por eso que algunos registros quedaron guardados de esta manera.

Sobre los lenguajes de programación utilizados algunos problemas se debieron a la sintaxis diferente entre lenguajes como lo son python y C#, además de encontrar fuentes confiables para el entendimiento de las librerías que se debían de utilizar para poder realizar lo solicitado con la base de datos, python y C# y para saber qué arquitectura se debe de utilizar para poder tener un proyecto hecho con buenas prácticas.

En resumen, los problemas encontrados fueron debido a la experiencia con estas nuevas herramientas para unos, y a la curva de aprendizaje para poder entender las nuevas herramientas que se utilizaron.

Conclusiones acerca del proyecto.

El proyecto propuesto por el maestro fue una gran elección para poder mejorar nuestras habilidades para el trabajo colaborativo, organización de tiempo, además de las habilidades y las herramientas que necesita un programador para poder trabajar en su día a día.

Fue una experiencia enriquecedora que nos ayudará de ahora en adelante a saber cómo poder interactuar con librerías, lenguajes y personas desconocidas, ya que para algunos de nosotros era la primera vez que interactuamos con tantos lenguajes y que además hicimos un proyecto colaborativo, a pesar de los problemas encontrados el proyecto se pudo realizar y se cumplieron los requisitos hablados desde el primer dia.